

Lekcja - przełożony sprawdzian LEKCJA jest zrobiona na 26 marca

Temat: is oraz ==, funkcja rekurencyjna. Typy mutowalne i niemutowalne, inkrementacja, dekrementacja. Try-except-else-finally, raise i custom exceptions

Operator **==** porównuje wartość. Inaczej mówiąc **sprawdza, czy wartości dwóch obiektów są takie same. Nie bierze pod uwagę, czy obiekty są przechowywane w tym samym miejscu w pamięci – liczy się tylko zawartość.** Jest to porównanie semantyczne, oparte na metodzie `__eq__` obiektu.

Przykłady

```
a = 5
b = 5
print(a == b)
```

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)
```

Operator **is** porównanie obiektu w pamięci. Inaczej mówiąc **sprawdza, czy dwie zmienne wskazują na ten sam obiekt w pamięci.**

Przykłady

```
a = [1, 2, 3]
b = [1, 2, 3]
print(a is b)
```

```
a = [1, 2, 3]
b = a
print(a is b)
```

Python ma mechanizm zwany **internowaniem małych liczb całkowitych** (integer caching).

- Dla liczb z zakresu **-5 do 256** Python **przechowuje jedną wspólną instancję**
- Czyli np. liczba 10 istnieje tylko raz w pamięci

```
a = 10
b = int("10")
print(a is b) # True
```

```
c = 256 # -5 do 256
d = int("256")
print(c is d) # True
```

```
e = 1000
f = int("1000")
print(e is f) # False
```

Przykład – list

```
a = [1, 2, 3]
b = [1, 2, 3]
```

```
print(a == b)
print(a is b)
```

Przykład – dict

```
a = {"name": "Jan"}
b = {"name": "Jan"}
```

```
print(a == b)
print(a is b)
```

Przykład – set

```
a = {1, 2, 3}
b = {1, 2, 3}
```

```
print(a == b)
print(a is b)
```

Przykład – is – None

```
x = None
```

```
if x is None:
    print("brak wartości")
```

Rekurencja to technika programowania, w której **funkcja wywołuje samą siebie, aby rozwiązać problem. Zamiast używać pętli** (jak for czy while), **funkcja dzieli problem na mniejsze podproblemy**, aż dojdzie do prostego przypadku, który można rozwiązać bezpośrednio. Kluczowe elementy funkcji rekurencyjnej:

- **Przypadek bazowy (base case):** Warunek, który kończy rekurencję. Bez niego funkcja będzie się wywoływać w nieskończoność, co spowoduje błąd (RecursionError w Pythonie, bo przekroczony zostanie limit rekurencji, domyślnie ok. 1000 wywołań).
- **Przypadek rekurencyjny (recursive case):** Część, w której funkcja wywołuje siebie z mniejszymi argumentami, zbliżając się do przypadku bazowego.

Rekurencja jest przydatna w problemach, które mają strukturę drzewiastą lub dzielą się na podproblemy, np. obliczanie silni, przeszukiwanie drzew, sortowanie (jak quicksort) czy generowanie fraktali.

Przykład: Obliczanie silni

```
def silnia(n):
    if n == 0 or n == 1: # Przypadek bazowy: 0! = 1, 1! = 1
        return 1
    else: # Przypadek rekurencyjny
        return n * silnia(n - 1)
```

Jak to działa?

- Dla **silnia(5)**: Wywołuje $5 * \text{silnia}(4)$
- **silnia(4)**: Wywołuje $4 * \text{silnia}(3)$
- **silnia(3)**: Wywołuje $3 * \text{silnia}(2)$
- **silnia(2)**: Wywołuje $2 * \text{silnia}(1)$
- **silnia(1)**: Zwraca **1** (przypadek bazowy)
- Teraz "wspinamy się" z powrotem: $2 * 1 = 2$, $3 * 2 = 6$, $4 * 6 = 24$, $5 * 24 = 120$.

Typy mutowalne i niemutowalne

W Pythonie typy danych dzielą się na **mutowalne (mutable)** i **niemutowalne (immutable)** w zależności od tego, czy można je modyfikować po utworzeniu. To kluczowa koncepcja, bo wpływa na to, jak działają przypisania, funkcje i struktury danych (np. listy jako klucze w słownikach nie działają, bo są mutowalne).

- **Niemutowalne (immutable)**: Po stworzeniu obiektu **nie można zmienić jego wartości. Zamiast modyfikować, Python tworzy nowy obiekt.** To bezpieczniejsze w kontekstach wielowątkowych i jako klucze w słownikach (muszą być hashable).
- **Mutowalne (mutable)**: **Można zmieniać zawartość obiektu bezpośrednio, bez tworzenia nowego.** To przydatne dla dynamicznych struktur, ale może prowadzić do nieoczekiwanych efektów (np. modyfikacja listy w funkcji wpływa na oryginalną listę).

Użyję tabeli do porównania powszechnych typów:

Typ danych	Mutowalny?	Przykłady użycia	Uwagi
int (liczba całkowita)	Nie	$x = 5$; $x = 6$ (tworzy nowy obiekt)	Zmiana wartości to nowe przypisanie.
float (liczba zmiennoprzecinkowa)	Nie	$y = 3.14$	Jak int, niezmienny.
str (ciąg znaków)	Nie	$s = \text{"hello"}$; $s.upper()$ (zwraca nowy string)	Metody jak <code>replace()</code> tworzą nowy obiekt.
tuple (krotka)	Nie	$t = (1, 2, 3)$	Nie można dodać/usunąć elementów.
frozenset (zamrożony zbiór)	Nie	$fs = \text{frozenset}([1, 2])$	Jak set, ale niezmienny.
list (lista)	Tak	$l = [1, 2]$; $l.append(3)$ (modyfikuje oryginalną)	Można dodawać, usuwać, sortować.

dict (słownik)	Tak	d = {'a': 1}; d['b'] = 2	Dodawanie/usuwanie kluczy/wartości.
set (zbiór)	Tak	s = {1, 2}; s.add(3)	Dynamiczne dodawanie/usuwanie.
bytearray (tablica bajtów)	Tak	ba = bytearray(b'abc'); ba[0] = 65	Modyfikowalna wersja bytes.

Różnice w praktyce

- **Przypisanie i modyfikacja:** Dla typów niemutowalnych, jak string, s = "hello"; s = s + " world" tworzy nowy string. Dla list: l = [1]; l += [2] modyfikuje istniejącą listę.
- **Funkcje i argumenty:** Jeśli przekażesz mutowalny obiekt do funkcji, zmiany w funkcji wpłyną na oryginalny obiekt (przekazywanie przez referencję). Dla niemutowalnych – nie.
- **Hashowalność:** Tylko niemutowalne typy mogą być kluczami w słownikach lub elementami w setach, bo ich hash nie zmienia się.

Przykład kodu

```
# Niemutowalny: string
s = "hello"
s2 = s # s2 wskazuje na ten sam obiekt
s = s.upper() # Tworzy nowy obiekt, s2 zostaje "hello"
print(s) # "HELLO"
print(s2) # "hello"
```

```
# Mutowalny: lista
l = [1, 2]
l2 = l # l2 wskazuje na tę samą listę
l.append(3) # Modyfikuje oryginalną listę
print(l) # [1, 2, 3]
print(l2) # [1, 2, 3] – też zmienione!
```

Python nie ma ++ i --

Python jest zaprojektowany, aby być prostym i czytelnym (zgodnie z "Zen of Python" – "Readability counts"). Operatory ++ i -- są skrótami, które mogą być mylące, bo działają różnie w zależności od kontekstu (np. pre-inkrementacja ++x vs. post-inkrementacja x++). **W Pythonie preferuje się jawne operacje, jak x = x + 1 lub x += 1, co jest łatwiejsze do zrozumienia na pierwszy rzut oka.**

```
x = 5
x++ # Błąd: SyntaxError: invalid syntax
```

W JavaScript np.:

Pre-inkrementacja/dekrementacja: Najpierw modyfikuje zmienną, potem zwraca nową wartość.

- ++x: Zwiększ x o 1, zwróć nowe x.
- --x: Zmniejsz x o 1, zwróć nowe x.

Post-inkrementacja/dekrementacja: Najpierw zwraca bieżącą wartość, potem modyfikuje zmienną.

- x++: Zwróć bieżące x, potem zwiększ o 1.

- `x--`: Zwróć bieżące `x`, potem zmniejsz o 1.

Python **nie ma wbudowanych operatorów `++` ani `--`**. Zamiast tego używa się operatorów **przypisania złożonego: `+=` dla inkrementacji i `-=` dla dekrementacji**. To proste i czytelne, ale wymaga jawnego zapisu. Te operatory działają na zmiennych liczbowych (`int`, `float`), ale też na niektórych kolekcjach (np. listy `z +=` do dodawania elementów).

Podstawowe przykłady

Inkrementacja

```
x = 5
x += 1  # To samo co x = x + 1
print(x)  # Wyjście: 6
```

Dekrementacja

```
y = 10
y -= 2  # To samo co y = y - 2
print(y)  # Wyjście: 8
```

Można używać z innymi wartościami

```
z = 3.5
z += 0.5  # Inkrementacja o 0.5
print(z)  # Wyjście: 4.0
```

Dla stringów lub list (rozszerzone użycie +=)

```
s = "Witaj"
s += " świecie!"  # Konkatenacja
print(s)  # Wyjście: Witaj świecie!
```

```
lista = [1, 2]
lista += [3]  # Dodawanie elementu
print(lista)  # Wyjście: [1, 2, 3]
```

Inkrementacja i dekrementacja często pojawiają się w pętlach, aby kontrolować liczniki lub iterować wstecz. Python preferuje pętle **for** z **`range()`**, które automatycznie obsługują inkrementację, ale w **while** musisz to robić ręcznie.

1. Pętla for z inkrementacją (używając `range`):

- a. `range(start, stop, step)` automatycznie inkrementuje (`step=1` domyślnie) lub dekrementuje (`step=-1`).

Inkrementacja w pętli for (od 0 do 4)

```
for i in range(5):  # i zaczyna od 0, inkrementuje o 1 co iterację
    print(i)  # Wyjście: 0 1 2 3 4
```

Dekrementacja w pętli for (od 5 do 1)

```
for j in range(5, 0, -1):  # Start=5, stop=0 (nie inkluzywny), step=-1
    print(j)  # Wyjście: 5 4 3 2 1
```

Ręczna inkrementacja w pętli for (rzadziej używana, ale możliwe)

```
licznik = 0
```

```
for _ in range(3): # Pętla 3 razy
    licznik += 2 # Inkrementacja o 2
    print(licznik) # Wyjście: 2 4 6
```

Pętla while z inkrementacją/dekrementacją:

- Tutaj operatory += i -= są kluczowe, bo kontrolujesz warunek ręcznie.

```
# Inkrementacja w while (licznik rosnący)
licznik = 0
while licznik < 5:
    print(licznik) # Wyjście: 0 1 2 3 4
    licznik += 1 # Inkrementacja
```

```
# Dekrementacja w while (licznik malejący)
licznik = 10
while licznik > 0:
    print(licznik) # Wyjście: 10 9 8 ... 1
    licznik -= 1 # Dekrementacja
```

```
# Przykład z warunkiem i inkrementacją o zmienną wartość
suma = 0
i = 1
while suma < 20:
    suma += i # Inkrementacja o i (1, potem 2, itd.)
    i += 1 # Inkrementacja i
    print(suma) # Wyjście: 1 3 6 10 15 21 (zatrzyma się przed 21, bo warunek)
```

Wyjątki w Pythonie: Try-except-else-finally, raise i custom exceptions

Wyjątki to mechanizm, który pozwala programowi radzić sobie z błędami w czasie wykonania (runtime errors), np. dzielenie przez zero, brak pliku czy nieoczekiwane dane. Zamiast crashować program, możesz "złapać" błąd i obsłużyć go elegancko. To kluczowe dla stabilnego kodu. Python ma wbudowane wyjątki (np. `ZeroDivisionError`, `FileNotFoundError`), ale możesz też tworzyć własne. Podstawowa struktura to blok try-except, z opcjonalnymi else i finally.

Podstawowa struktura: try-except

- try:** W tym bloku umieszczasz kod, który może spowodować błąd.
- except:** Łapie wyjątek i obsługuje go. Możesz sprecyzować typ wyjątku (np. `except ValueError:`) lub złapać wszystkie (`except:` – ale to niezalecane, bo maskuje błędy).

Przykład

```
try:
    liczba = int(input("Podaj liczbę: ")) # Może rzucić ValueError, jeśli nie liczba
    wynik = 10 / liczba # Może rzucić ZeroDivisionError
```

```

    print(wynik)
except ValueError:
    print("To nie jest liczba!")
except ZeroDivisionError:
    print("Nie dziel przez zero!")

```

Dodatkowe bloki: else i finally

- **else:** Wykonuje się tylko, jeśli w try NIE wystąpił żaden wyjątek. Przydatne do kodu, który ma działać po sukcesie.
- **finally:** Zawsze się wykonuje, niezależnie od tego, czy był wyjątek czy nie. Idealne do czyszczenia zasobów (np. zamykanie plików).

```

try:
    a = int(input("Podaj pierwszą liczbę: ")) # Może rzucić ValueError
    b = int(input("Podaj drugą liczbę: ")) # Może rzucić ValueError
    wynik = a / b # Może rzucić ZeroDivisionError
except ValueError:
    print("Jedna z wartości nie jest liczbą całkowitą!")
except ZeroDivisionError:
    print("Nie można dzielić przez zero!")
except Exception as e:
    print(f"Nieoczekiwany błąd: {e}")
else:
    print(f"Wynik dzielenia: {wynik}")
    print("Obliczenia zakończone sukcesem.")
finally:
    print("Program zakończył przetwarzanie wejścia – zawsze to się wyświetli.")

```

Raise: Rzucanie wyjątków raise pozwala ręcznie "rzucić" wyjątek, np. gdy chcesz przerwać wykonanie przy niepoprawnych danych. Możesz raise'ować wbudowany wyjątek lub własny.

Przykład:

```

def sprawdz_wiek(wiek):
    if wiek < 18:
        raise ValueError("Jesteś za młody!") # Rzuca wyjątek z komunikatem
    return "OK"

try:
    sprawdz_wiek(15)
except ValueError as e:
    print(e) # Wyjście: Jesteś za młody!

```

Custom exceptions: Własne wyjątki Możesz tworzyć własne klasy wyjątków, dziedzicząc po Exception (lub podklasach jak ValueError). To przydatne w dużych projektach, by mieć specyficzne błędy. Przykład tworzenia i użycia:

```

class MojBład(Exception): # Dziedziczy po Exception
    def __init__(self, wiadomosc="To mój custom błąd!"):
        self.wiadomosc = wiadomosc
        super().__init__(self.wiadomosc)

def funkcja():
    raise MojBład("Coś poszło nie tak.")

try:
    funkcja()
except MojBład as e:
    print(e) # Wyjście: To mój custom błąd!

```

Lekcja

Temat: Metody dla list (list), zbiorów (set) i słowników (dict)

Lista to **uporządkowana kolekcja elementów**, która **pozwalą na duplikaty** i ma **indeksy**.

```
numbers = [1, 2, 3]
```

append(x)

Dodaje element **na koniec listy**.

```
numbers.append(4)
print(numbers) # [1,2,3,4]
```

extend(iterable)

Dodaje **wiele elementów naraz**.

```
numbers.extend([5,6])
print(numbers) # [1,2,3,4,5,6]
```

insert(index, x)

Wstawia element w określone miejsce.

```
numbers.insert(1, 10)
print(numbers) # [1,10,2,3]
```

remove(x)

Usuwa **pierwsze wystąpienie elementu**. Jeśli element nie istnieje, zwraca błąd.

```
numbers.remove(2)
print(numbers) # [1,3]
```

pop([index])

Usuwa element i **zwraca go**.

```
numbers = [1,2,3,4]
last = numbers.pop()      # usuwa ostatni
print(last)               # 4
removedElementAtIndex = numbers.pop(1)  # usuwa element z indeksu
print(removedElementAtIndex)           # 2
print(numbers)                       # [1, 3]
```

clear()

Usuwa wszystkie elementy.

```
numbers.clear()
print(numbers)      # []
```

index(x)

Zwraca indeks pierwszego wystąpienia.

```
numbers = [1,2,3,4]
i=numbers.index(3)
print(i)          # 2
```

count(x)

Liczy ile razy element występuje.

```
numbers = [1,1,2,3]
c=numbers.count(1)
print(c)          # 2
```

sort()

Sortuje listę.

```
numbers = [3,1,2]
numbers.sort()
print(numbers)      # [1,2,3]
```

reverse()

Odwraca kolejność.

```
numbers = [3,1,2]
numbers.reverse()
print(numbers)      # [2, 1, 3]
```

copy()

Tworzy kopię listy.

```
numbers = [3,1,2]
new_list = numbers.copy()
numbers.append(6)
new_list.append(5)
print(numbers)      # [3, 1, 2, 6]
print(new_list)      # [3, 1, 2, 5]
```

Set to nieuporządkowany zbiór bez duplikatów.

$A = \{1,2,3\}$

add(x)

Dodaje element. **Element 4 zostaje dodany do zbioru, ale nie wiadomo w jakiej kolejności.** Zbiór **nie ma uporządkowania**, więc nie można powiedzieć, że element jest „na końcu” ani „na początku”.

```
A = {1,2,3}
A.add(4)
print(A)           # {1, 2, 3, 4}
```

update(iterable)

Dodaje wiele elementów. **Elementy zostaną dodane do zbioru, jeśli ich tam jeszcze nie było. Nie wiadomo w jakiej kolejności się pojawią przy wyświetleniu.**

```
A = {1, 2, 3}
A.update({7, 8})    # set
A.update((9, 10))   # tuple
A.update("ab")      # string → dodaje 'a' i 'b'
print(A)           # {1, 2, 3, 7, 8, 9, 10, 'b', 'a'}
```

remove(x)

Usuwa element (błąd jeśli nie istnieje).

```
A = {1, 2, 3}
A.remove(2)
print(A)           # {1, 3}
```

discard(x)

Usuwa element **bez błędu**.

```
A = {1, 2, 3}
A.discard(4)
print(A)           # {1, 2, 3}
```

pop()

Usuwa **losowy element**. **Usuwa i zwraca usunięty jeden element ze zbioru. Nie wiadomo, który element zostanie usunięty, bo set jest nieuporządkowany.**

```
A = {1, 2, 3, 4}
x = A.pop()
print(x)           # 1
print(A)           # {2, 3, 4}
```

clear()

Czyści zbiór.

```
A.clear()
print(A)           # set()
```

copy()

```
A = {1, 2, 3, 4}
B = A.copy()
A.add(6)
B.add(7)
print(A)           # {1, 2, 3, 4, 6}
print(B)           # {1, 2, 3, 4, 7}
```

Operacje zbiorów

union()

Łączy zbiory.

A = {1,2}

B = {2,3}

```
print(A.union(B))    # {1,2,3}
```

intersection()

Część wspólna.

A = {1,2}

B = {2,3}

```
print(A.intersection(B)) # {2}
```

difference()

Różnica zbiorów.

A = {1,2}

B = {2,3}

```
print(A.difference(B))    # {1}
```

symmetric_difference()

Zwraca **nowy zbiór zawierający elementy, które są w **A lub w B**, ale **nie w obu naraz**.

Formuła matematyczna:

$$A \triangle B = (A - B) \cup (B - A)$$

A = {1,2}

B = {2,3}

```
print(A.symmetric_difference(B)) # {1,3}
```

issubset()

A.issubset(B)

- **Zwraca True**, jeśli każdy element zbioru A jest też w zbiorze B.
- **Zwraca False**, jeśli choć jeden element A nie występuje w B.

Przykład

A = {1, 2}

B = {1, 2, 3, 4}

```
print(A.issubset(B))    # True
```

issuperset()

A.issuperset(B)

- Zwraca **True**, jeśli **wszystkie elementy zbioru B są też w A**.
- Zwraca **False**, jeśli choć jeden element B nie występuje w A.

Innymi słowy: A jest **nadzbiorem** B.

Przykład

A = {1, 2, 3, 4}

```
B = {2, 3}
print(A.issuperset(B))    # True
```

isdisjoint()

```
A.isdisjoint(B)
```

```
A.isdisjoint(B)
```

- Zwraca **True**, jeśli **żaden element A nie występuje w B**.
- Zwraca **False**, jeśli jest **przynajmniej jeden wspólny element**.

Sprawdza czy zbiory nie mają części wspólnej.

Przykład – brak wspólnych elementów

```
A = {1, 2, 3}
```

```
B = {4, 5, 6}
```

```
print(A.isdisjoint(B)) # True
```

DICTIONARY (dict) Słownik to para klucz → wartość.

Podstawowy obiekt

```
person = {
    "name": "Jan",
    "age": 30
}
```

get(key)

Pobiera wartość bez błędu.

```
person = {
    "name": "Jan",
    "age": 30
}
```

```
print(person.get("name"))
```

keys()

```
print(person.keys())    # dict_keys(['name', 'age']) zwraca wszystkie klucze.
```

values()

```
print(person.values())  # dict_values(['Jan', 30]) zwraca wartości.
```

items()

```
print(person.items())   # dict_items([('name', 'Jan'), ('age', 30), ('city', 'Warsaw')])
zwraca pary (key,value).
```

update()

```
person = {
    "name": "Jan",
    "age": 30
}
person.update({"city": "Warsaw"})
```

```
print(person) # {'name': 'Jan', 'age': 30, 'city': 'Warsaw'}
```

pop(key)

usuwa element

```
person = {
    "name": "Jan",
    "age": 30
}
person.pop("age")
print(person)      # {'name': 'Jan'}
```

popitem()

usuwa **ostatnią** i zwraca co usuwa

```
person = {
    "name": "Jan",
    "first": "Kowalski",
    "age": 30,
    "gender": "M"
}

p = person.popitem()
print(p)      # ('gender', 'M')
print(person) # {'name': 'Jan', 'first': 'Kowalski', 'age': 30}
```

setdefault()

Dodaje klucz jeśli nie istnieje.

```
person = {
    "name": "Jan",
    "age": 30
}

person.setdefault("country", "Poland")
print(person)      # {'name': 'Jan', 'age': 30, 'country': 'Poland'}
```

clear()

```
person.clear()
print(person)      # {}
```

copy()

```
person = {"name": "Jan", "age": 30, "city": "Warsaw"}

new_person = person.copy()
person.update({"city": "Warsaw2"})
new_person.update({"post-code": "12-333"})
```

```
print(person)      # {'name': 'Jan', 'age': 30, 'city': 'Warsaw2'}
print(new_person)  # {'name': 'Jan', 'age': 30, 'city': 'Warsaw', 'post-code': '12-333'}
```

new_person to **nowy słownik**. Zmiany w new_person **nie wpływają na person**, jeśli zmieniasz wartości bezpośrednio:

```
new_person["age"] = 31
print(person)      # {'name': 'Jan', 'age': 30, 'city': 'Warsaw'}
print(new_person)  # {'name': 'Jan', 'age': 31, 'city': 'Warsaw'}
```

Lekcja

Temat: Brak modyfikatorów dostępu

W Pythonie nie ma prawdziwych modyfikatorów dostępu (jak *public*, *private*, *protected* w *Java/C++/C#*). Nie istnieją żadne słowa kluczowe do tego celu.

Python używa tylko **konwencji nazewnictwa** (i jednego mechanizmu name mangling). Są dokładnie trzy poziomy:

Poziom	Zapis	Co oznacza	Czy naprawdę chroniony?
Public	<code>self.nazwa</code>	Domyślny – każdy może czytać i modyfikować	Nie – całkowicie otwarty
Protected	<code>self._nazwa</code>	Tylko konwencja („nie ruszaj z zewnątrz klasy”)	Nie – dostępny zawsze
Private	<code>self.__nazwa</code>	Name mangling + konwencja	Prawie tak (ale da się obejść)

- **Public** – bez podkreślnika
Wszystko, co nie ma _ na początku, jest publiczne.
- **Protected** – pojedynczy _
Tylko sygnalizuje innym programistom: „to jest wewnętrzne, lepiej nie używaj bezpośrednio”. Python **nie blokuje** dostępu. W Pythonie **protected** to **tylko konwencja**, nie prawdziwa ochrona.
- **Private** – dwa _
Python automatycznie zmienia nazwę na `_NazwaKlasy__nazwa` (name mangling). Dzięki temu nie da się przypadkowo odwołać z zewnątrz, ale **nadal można** dostać się przez `_Klasa__nazwa`. To jest **najmocniejsza forma "prywatności"** jaką Python oferuje. Python robi coś specjalnego nazywanego **name mangling** („przekształcanie nazwy”).

Przykład:

```
class Osoba:
    def __init__(self, imie, wiek):
        self.imie = imie          # ← PUBLICZNA (self.nazwa)
        self._wiek = wiek         # ← protected (tylko dla porównania)
        self.__haslo = "sekret"   # ← private (tylko dla porównania)

    def przedstaw_sie(self):
        print(f"Cześć, jestem {self.imie}")
```

```
class Raport:

    def generuj_raport(self, osoba: Osoba):

        print(f"RAPORT:")
        print(f"    Imię: {osoba.imie}")          # ← działa zawsze
        # print(osoba._wiek)                     # działa, ale nie powinienieś
        # print(osoba.__haslo)                   # AttributeError
```

```
class Pracownik(Osoba):
    def __init__(self, imie, wiek, stanowisko):
        super().__init__(imie, wiek)
        self.stanowisko = stanowisko

    def pokaz_dane(self):

        print(f"Pracownik: {self.imie}")
        print(f"Stanowisko: {self.stanowisko}")
```

```
jan = Osoba("Jan Kowalski", 35)
anna = Pracownik("Anna Nowak", 28, "programista")
```

```
raport = Raport()
raport.generuj_raport(jan)
raport.generuj_raport(anna)
```

```
anna.pokaz_dane()
```

```
print(f"Bezpośrednio: {jan.imie}")
```

Dlaczego nie stosować **Protected** – pojedynczy _

1. Wyobraź sobie, że klasa `Osoba` jest napisana przez kogoś innego (np. bibliotekę, kolegę z zespołu, albo Ciebie sprzed 6 miesięcy).

```
class Osoba:
    def __init__(self, imie, wiek):
        self.imie = imie
        self._wiek = wiek          # ← PROTECTED (tylko konwencja!)

    def urodziny(self):
        self._wiek += 1

    def pokaz_wiek(self):
        print(f"{self.imie} ma {self._wiek} lat")
```

2. Co się stanie, jeśli Ty użyjesz `osoba._wiek` z zewnątrz?

```
jan = Osoba("Jan", 30)
print(jan._wiek)          # 30 ← działa
jan._wiek = 100           # zmieniamy bezpośrednio ← ZŁE!
```

To działa... ale jest **niebezpieczne**.

3. Za miesiąc autor klasy `Osoba` robi update:

```
from datetime import date
class Osoba:
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # ← nadal protected
        # self._wiek już NIE ISTNIEJE!

    def urodziny(self):
        print(f"🎉 Wszystkiego najlepszego {self.imie}!")

    @property
    def wiek(self):
        today = date.today()
        wiek = today.year - self._data_urodzenia.year
        if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):
            wiek -= 1
        return wiek
```

Co się stało z kodem, który używał `jan._wiek`?

```
jan = Osoba("Jan", date(1995, 3, 15))
print(jan._wiek)          # ← AttributeError! Nie ma już takiej zmiennej!
jan._wiek = 50            # ← też błąd!
```


4. TAK POWINNO SIĘ ROBIĆ

```
from datetime import date

class Osoba:
    def __init__(self, imie, data_urodzenia):
        self.imie = imie
        self._data_urodzenia = data_urodzenia    # protected – tylko wewnątrz

    def urodziny(self):
        print(f"🎂 Wszystkiego najlepszego {self.imie}!")

    @property
    def wiek(self):
        """Zwraca aktualny wiek"""
        today = date.today()
        wiek = today.year - self._data_urodzenia.year
        if (today.month, today.day) < (self._data_urodzenia.month,
self._data_urodzenia.day):
            wiek -= 1
        """
        today = (3, 10)
        urodziny = (3, 15)

        (3, 10) < (3, 15) → True
        """
        return wiek

    @wiek.setter
    def wiek(self, nowy_wiek):
        """setter – bezpieczna zmiana"""
        if nowy_wiek < 0:
            raise ValueError("Wiek nie może być ujemny!")
        # przeliczamy datę urodzenia na podstawie nowego wieku
        today = date.today()
        self._data_urodzenia = date(today.year - nowy_wiek, today.month, today.day)

jan = Osoba("Jan Kowalski", date(1995, 3, 15))

print(jan.wiek)    # 30 (lub aktualny wiek) ← OK
jan.wiek = 31      # zmienia wiek bezpiecznie ← OK
jan.urodziny()

# jan._data_urodzenia    # ← działa technicznie, ale NIE RÓB TEGO!
```

Przykład Private – dwa podkreślniki __

```
class KontoBankowe:
    def __init__(self, wlasciciel):
        self.wlasciciel = wlasciciel
        self.__saldo = 5000          # ← PRIVATE (dwa __)

    def wpłac(self, kwota):
        self.__saldo += kwota
        print(f"Wpłacono {kwota} zł")

    def wypłac(self, kwota):
        if kwota <= self.__saldo:    # używamy wewnątrz klasy – OK
            self.__saldo -= kwota
            print(f"Wypłacono {kwota} zł")
        else:
            print("Brak środków!")

    def pokaz_saldo(self):
        print(f"Saldo: {self.__saldo} zł")    # też wewnątrz klasy

konto = KontoBankowe("Jan Kowalski")

konto.wpłac(1000)
konto.pokaz_saldo()          # działa → Saldo: 6000 zł

# Próba bezpośredniego dostępu (to, co większość ludzi próbuje):
print(konto.__saldo)          # ← AttributeError: 'KontoBankowe' object has no attribute
                              # '.__saldo'

# Próba zmiany:
# konto.__saldo = 999999      # też nie działa tak, jak myślisz
```

Jak obejść name mangling?

```
# Prawdziwa nazwa po manglingu:
print(konto._KontoBankowe__saldo)    # ← działa! → 6000

# Możemy nawet zmienić:
konto._KontoBankowe__saldo = 100     # teraz saldo = 100 zł
konto.pokaz_saldo()                  # → Saldo: 100 zł
```

Lekcja

Temat: Polimorfizm i enkapsulacja

Python jest językiem obiektowym i w pełni wspiera polimorfizm oraz enkapsulację (hermetyzację).

Polimorfizm w Pythonie to zdolność obiektów różnych klas do reagowania na te same metody w sposób odpowiedni dla ich typu. Oznacza to, że ta sama operacja może zachowywać się inaczej w zależności od typu obiektu, na którym jest wykonywana.

1. Polimorfizm funkcji wbudowanych

```
# Funkcja len() działa z różnymi typami
print(len("Hello")) # 5 (string)
print(len([1, 2, 3, 4])) # 4 (lista)
print(len({'a': 1, 'b': 2})) # 2 (słownik)
```

2. Polimorfizm z klasami i metodami

Polimorfizm - ta sama metoda, różne zachowania

```
class Pies:
    def dzwiek(self):
        return "Hau hau!"

    def opis(self):
        return "Jestem psem"

class Kot:
    def dzwiek(self):
        return "Miau!"
    def opis(self):
        return "Jestem kotem"

class Krowa:
    def dzwiek(self):
        return "Muuu!"
    def opis(self):
        return "Jestem krową"

def wydaj_dzwiek(zwierze):
    print(zwierze.dzwiek())

pies = Pies()
kot = Kot()
krowa = Krowa()
```

```
wydaj_dzwiek(pies)
wydaj_dzwiek(kot)
wydaj_dzwiek(krowa)
```

3. Polimorfizm z dziedziczeniem

1. Przykład

```
class Figura:
    def pole(self):
        pass

    def obwod(self):
        pass

class Prostokat(Figura):
    def __init__(self, szerokosc, wysokosc):
        self.szerokosc = szerokosc
        self.wysokosc = wysokosc

    def pole(self):
        return self.szerokosc * self.wysokosc

    def obwod(self):
        return 2 * (self.szerokosc + self.wysokosc)

class Kolo(Figura):
    def __init__(self, promien):
        self.promien = promien

    def pole(self):
        return 3.14 * self.promien ** 2

    def obwod(self):
        return 2 * 3.14 * self.promien

class Trojkat(Figura):
    def __init__(self, a, b, c):
        self.a = a
        self.b = b
        self.c = c

    def pole(self):
        # Wzór Herona
        s = (self.a + self.b + self.c) / 2
        return (s * (s - self.a) * (s - self.b) * (s - self.c)) ** 0.5

    def obwod(self):
        return self.a + self.b + self.c

# Użycie polimorfizmu
figury = [ Prostokat(5, 10), Kolo(7), Trojkat(3, 4, 5) ]
for figura in figury:
    print(f"Pole: {figura.pole():.2f}")
    print(f"Obwód: {figura.obwod():.2f}")
```

```
print("---")
```

Wyjście:

```
Pole: 50.00
Obwód: 30.00
---
Pole: 153.86
Obwód: 43.96
---
Pole: 6.00
Obwód: 12.00
---
```

2. Przykład

```
class Kształt:
    def __init__(self, nazwa= " Kształt "):
        self.nazwa = nazwa

    def pole(self):
        print(" Brak danych na temat ", self.nazwa)

class Trojkat(Kształt):
    def __init__(self, nazwa= " Trojkat ", a=2, h=2):
        super().__init__(nazwa)
        self.a = a
        self.h = h

    def pole(self):
        print(" Pole figury ", self.nazwa)
        print(self.a* self.h/2)

class Prostokat(Kształt):
    def __init__(self, nazwa= " Prostokat ", a=2, b=2):
        super().__init__(nazwa)
        self.a = a
        self.b = b

    def pole(self):
        print(" Pole figury ", self.nazwa)
        print(self.a* self.b)

class Kwadrat():
    def __init__(self, nazwa="Kwadrat", a=3):
        self.nazwa = nazwa
        self.a = a

    def pole(self):
        print("Pole figury ", self.nazwa)
        print(self.a**2)

def wyswietl_pole(lista):
    for x in lista:
        x.pole()
```

```
ksztalt = Kszalt()
trojkat = Trojkat()
prostokat = Prostokat()
kwadrat = Kwadrat()
```

```
for x in (ksztalt, trojkat, prostokat, kwadrat):
    print(type(x))
    x.pole()
```

4. Polimorfizm z operatorami (przeciążanie operatorów)

Przeciążenie operatorów (Operator Overloading)

Przeciążenie operatorów to mechanizm, dzięki któremu możemy **zmienić zachowanie** standardowych operatorów (+, -, *, /, ==, >, [] itd.) dla naszych własnych klas.

Dzięki temu zamiast pisać:

```
v3 = v1.add(v2)
```

możemy pisać naturalnie:

```
v3 = v1 + v2
```

Python umożliwia to poprzez **specjalne metody** (tzw. **dunder methods** – double underscore), które zaczynają i kończą się na `__`.

Przykład

`class` **Wektor**:

```
def __init__(self, x, y):
    self.x = x
    self.y = y

def __add__(self, other):
    # Przeciążenie operatora +
    return Wektor(self.x + other.x, self.y + other.y)

def __str__(self):
    return f"Wektor({self.x}, {self.y})"

def __mul__(self, scalar):
    # Mnożenie przez skalar
    return Wektor(self.x * scalar, self.y * scalar)
```

```
v1 = Wektor(2, 3)
v2 = Wektor(5, 7)
v3 = v1 + v2 # Używamy przeciążonego operatora +
```

```
print(v3) # Wektor(7, 10)
print(v1 * 3) # Wektor(6, 9)
```

Pod spodem

- $v1 + v2 \rightarrow$ Python szuka metody `__add__` w klasie `Wektor`
- `print(v3)` $\rightarrow$ Python szuka metody `__str__`
- $v1 * 3 \rightarrow$ Python szuka metody `__mul__`

Najważniejsze metody do przeciążania operatorów

Operator	Metoda specjalna	Przykład użycia
+	<code>__add__</code>	<code>v1 + v2</code>
-	<code>__sub__</code>	<code>v1 - v2</code>
*	<code>__mul__</code>	<code>v1 * 3</code>
/	<code>__truediv__</code>	<code>v1 / 2</code>
//	<code>__floordiv__</code>	<code>v1 // 2</code>
==	<code>__eq__</code>	<code>v1 == v2</code>
!=	<code>__ne__</code>	<code>v1 != v2</code>
>	<code>__gt__</code>	<code>v1 > v2</code>
<	<code>__lt__</code>	<code>v1 < v2</code>
>=	<code>__ge__</code>	<code>v1 >= v2</code>
<=	<code>__le__</code>	<code>v1 <= v2</code>
<code>len()</code>	<code>__len__</code>	<code>len(v1)</code>
<code>str()</code> / <code>print</code>	<code>__str__</code>	<code>print(v1)</code>
<code>repr()</code>	<code>__repr__</code>	<code>repr(v1)</code>
<code>[]</code>	<code>__getitem__</code>	<code>v1[0]</code>

5. Polimorfizm operatorów

Przykład

```
print(2 + 3) # 5
print("2" + "3") # "23"
```

Główne zastosowania polimorfizmu:

1. **Uproszczenie kodu** - jedna funkcja obsługuje wiele typów obiektów
2. **Rozszerzalność** - łatwo dodawać nowe typy bez zmiany istniejącego kodu
3. **Abstrakcja** - ukrywanie szczegółów implementacji
4. **Zgodność z zasadą Open/Closed** - otwarty na rozszerzenia, zamknięty na modyfikacje
5. **Czytelność** - kod jest bardziej intuicyjny i łatwiejszy do zrozumienia

Enkapsulacja w Pythonie to mechanizm ukrywania wewnętrznych szczegółów implementacji klasy i udostępniania tylko tego, co jest niezbędne dla użytkownika. Chroni dane przed bezpośrednim dostępem i modyfikacją z zewnątrz.

Poziomy dostępu w Pythonie:

1. Publiczne (public) - brak podkreślników

```
class Osoba:
    def __init__(self, imie):
        self.imie = imie # Publiczne - dostępne wszędzie

    def przedstaw_sie(self): # Publiczna metoda
        return f"Cześć, jestem {self.imie}"

osoba = Osoba("Anna")
print(osoba.imie) # Anna - bezpośredni dostęp OK
print(osoba.przedstaw_sie()) # Cześć, jestem Anna
```

2. Chronione (protected) - jeden podkreślnik _

```
class KontoBankowe:
    def __init__(self, wlasciciel, saldo):
        self.wlasciciel = wlasciciel
        self._saldo = saldo # Chronione - konwencja, nie dla bezpośredniego użycia
    def _waliduj_kwota(self, kwota): # Chroniona metoda
        return kwota > 0

    def wplata(self, kwota):
        if self._waliduj_kwota(kwota):
            self._saldo += kwota
            return True
        return False

    def pobierz_saldo(self):
        return self._saldo

konto = KontoBankowe("Jan", 1000)
print(konto.pobierz_saldo()) # 1000 - OK przez metodę publiczną

# Techniczne możliwe, ale nie zalecane (konwencja):
print(konto._saldo) # 1000 - działa, ale narusza konwencję
```

3. Prywatne (private) - dwa podkreślniki __

```
class Sejf:
    def __init__(self, kod_pin):
        self.__kod_pin = kod_pin # Prywatne - name mangling
        self.__zawartosc = []
```



```

def __sprawdz_kod(self, kod): # Prywatna metoda
    return kod == self.__kod_pin

def otworz(self, kod):
    if self.__sprawdz_kod(kod):
        return f"Sejf otwarty. Zawartość: {self.__zawartosc}"
    return "Błędny kod PIN!"

def dodaj_przedmiot(self, kod, przedmiot):
    if self.__sprawdz_kod(kod):
        self.__zawartosc.append(przedmiot)
        return "Dodano przedmiot"
    return "Błędny kod PIN!"

sejf = Sejf("1234")
print(sejf.otworz("1234")) # Działa przez publiczną metodę

# To NIE ZADZIAŁA:
# print(sejf.__kod_pin) # AttributeError
# sejf.__sprawdz_kod("1234") # AttributeError

# Name mangling - można obejść, ale NIE POWINNO SIĘ:
# print(sejf._Sejf__kod_pin) # 1234 - możliwe, ale złamanie enkapsulacji

```

4. Właściwości (Properties) - gettery i settery

W wielu językach (Java, C#) pisze się:

```

getPensja()    // getter
setPensja(5000) // setter

```

W Pythonie robi się to zupełnie inaczej – i jest to **świadoma, filozoficzna decyzja** języka.

Główny powód: Pythonic way – „Uniform Access Principle”

Zasada jednolitego dostępu mówi:

Klient klasy **nie powinien widzieć różnicy** między:

- zwykłym atrybutem (`pracownik.pensja`)
- a obliczaną/walidowaną wartością (`pracownik.pensja`)

Dzięki `@property` możesz **zmienić implementację** w przyszłości **bez zmiany kodu**, który tej klasy używa.

Porównanie: bez property vs z property

1. Wersja „stara”

```
class Pracownik:
    def __init__(self, imie, pensja):
        self.__imie = imie
        self.__pensja = pensja

    def get_pensja(self):
        return self.__pensja

    def set_pensja(self, nowa_pensja):
        if nowa_pensja < 0:
            raise ValueError("Pensja nie może być ujemna!")
        self.__pensja = nowa_pensja

p = Pracownik("Marek", 5000)
print(p.get_pensja())      # trzeba pamiętać o nawiasach i "get_"
p.set_pensja(6000)        # trzeba pamiętać o "set_"
```

2. Wersja z @property

```
class Pracownik:
    def __init__(self, imie, pensja):
        self.__imie = imie
        self.__pensja = pensja

    @property
    def pensja(self):
        return self.__pensja

    @pensja.setter
    def pensja(self, nowa_pensja):
        if nowa_pensja < 0:
            raise ValueError("Pensja nie może być ujemna!")
        if nowa_pensja < 3000:
            raise ValueError("Pensja nie może być niższa niż minimalna!")
        self.__pensja = nowa_pensja

p = Pracownik("Marek", 5000)
print(p.pensja)            # wygląda jak zwykły atrybut!
p.pensja = 6000            # też wygląda jak zwykły atrybut!
```

Kluczowa korzyść:

Jeśli w przyszłości dodasz walidację, logowanie, przeliczanie waluty itp. – **nie musisz zmieniać żadnego wiersza kodu** u użytkowników Twojej klasy.

Lekcja

Temat: Obsługi plików: Otwieranie (with open), czytanie/zapis (read, write), tryby (r, w, a).

Obsługa plików

Python ma bardzo prosty i **bezpieczny sposób** pracy z plikami dzięki wbudowanej funkcji `open()` oraz **menedżerowi kontekstu** `with`. Dzięki temu **nie musisz pamiętać o ręcznym zamykaniu pliku** (`close()`), a kod jest czytelniejszy i mniej podatny na błędy.

1. Otwieranie pliku – with open(...)

Zalecany sposób:

```
with open('nazwa_pliku.txt', 'tryb') as plik:
    # tutaj pracujemy z plikiem
    # po wyjściu z bloku with plik jest automatycznie zamykany
```

with

- Automatycznie zamyka plik nawet jeśli wystąpi błąd (wyjątek).
- Kod jest krótszy i czytelniejszy.
- Nie musisz pamiętać o `plik.close()`.

2. Tryby otwierania plików (mode)

'r' – read (domyślny)

Otwiera plik tylko do odczytu. Plik **musi istnieć**, inaczej dostaniesz błąd.

'w' – write

Tworzy nowy plik lub **całkowicie nadpisuje** istniejący.

'a' – append

Dopisuje dane na końcu pliku. Jeśli plik nie istnieje – automatycznie go tworzy.

Dodatkowo często spotykane:

- 'r+' – odczyt + zapis
- 'w+' – odczyt + zapis (nadpisuje)
- 'a+' – odczyt + dopisywanie

Przykład otwierania w różnych trybach:

```
with open('dane.txt', 'r') as plik:      # tylko odczyt
    ...

with open('wyniki.txt', 'w') as plik:    # nadpisanie
    ...

with open('log.txt', 'a') as plik:      # dopisywanie
    ...
```

3. Czytanie pliku – metody read(), readline(), readlines()

Przykład - czytanie całego pliku:

```
with open('dane.txt', 'r', encoding='utf-8') as plik:
    # 1. Odczyt całego pliku naraz (najprostsze)
    zawartosc = plik.read()
    print(zawartosc)

    # 2. Odczyt linia po linii (najlepsze dla dużych plików)
    plik.seek(0) # wróć na początek pliku
    for linia in plik:
        print(linia.strip())          # strip() usuwa \n na końcu

    plik.seek(0)
    # 3. Odczyt jako lista linii
    linie = plik.readlines()
    print(linie)
```

Praktyczne przykłady zastosowania:

```
# Przykład 1: Wczytanie konfiguracji
'''
Zawartość pliku config.txt:
host = localhost
port = 8080
username = admin
password = sekret
debug = true
timeout = 30
'''
```

```
'''
# x.strip() usuwa białe znaki z początku i końca napisu (czyli spacje i \n).

with open('config.txt', 'r', encoding='utf-8') as plik:
    for linia in plik:
        if '=' in linia:
            klucz, wartosc = [x.strip() for x in linia.split('=', 1)]
            print(f"{klucz} = {wartosc}")
'''

Zawartość pliku zakupy.txt:
mleko
chleb
jajka
masło
ser
'''

# Przykład 2: Wczytanie listy zakupów do listy Pythona
with open('zakupy.txt', 'r', encoding='utf-8') as plik:
    lista_zakupow = [linia.strip() for linia in plik if linia.strip()]
    print(lista_zakupow)
```

4. Zapis pliku – metoda write() i writelines()

```
# Tryb 'w' → nadpisuje plik
with open('wyniki.txt', 'w', encoding='utf-8') as plik:
    plik.write("Wynik obliczeń: 42\n")
    plik.write("Druga linia tekstu\n")

# Można też zapisać wiele linii naraz
linie = ["Pierwsza\n", "Druga\n", "Trzecia\n"]
plik.writelines(linie)
```

Zapisze plik wyniki.txt z taką zawartością:

```
Wynik obliczeń: 42
Druga linia tekstu
Pierwsza
Druga
Trzecia
```

```
# Tryb 'a' → dopisuje na końcu (idealny do logów)
import datetime
```

```
with open('log.txt', 'a', encoding='utf-8') as plik:
    czas = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    plik.write(f"[{czas}] Użytkownik załogował się\n")
    plik.write(f"[{czas}] Błąd: coś poszło nie tak\n")
```

Praktyczne przykłady zastosowania:

```
import datetime
# Przykład 1: Zapis listy zakupów (nadpisanie)
zakupy = ["mleko", "chleb", "masło", "jajka"]
```

```
with open('lista_zakupow.txt', 'w', encoding='utf-8') as plik:
    for produkt in zakupy:
        plik.write(produkt + '\n')
```

Przykład 2: Logowanie błędów (dopisywanie)

```
try:
    1 / 0
except Exception as e:
    with open('errors.log', 'a', encoding='utf-8') as plik:
        plik.write(f"[{datetime.datetime.now()}] BŁĄD: {e}\n")
```

Wynik:

Utworzy plik `lista_zakupow.txt` z zawartością:

mleko

chleb

masło

jajka

Lekcja - Zapowiedzieć kartkówkę

Temat: List comprehensions, generatory i wyrażenia generatorowe. Match

1. List comprehensions (wyrażenia listowe)

To najwygodniejszy i najbardziej „pythoniczny” sposób tworzenia list w jednej linii.

Składnia podstawowa:

[wyrażenie **for** element **in** iterable **if** warunek]

Przykłady:

Prosty przykład

```
kwadraty = [x**2 for x in range(10)]
print(kwadraty)          # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

Z warunkiem (tylko parzyste)

```
parzyste_kwadraty = [x**2 for x in range(10) if x % 2 == 0]
print(parzyste_kwadraty) # [0, 4, 16, 36, 64]
```

Złożone wyrażenie

```
imiona = ["ania", "tomek", "natalia", "piotr"]
duze_imiona = [imie.capitalize() for imie in imiona if len(imie) > 3]
print(duze_imiona)          # ['Ania', 'Tomek', 'Natalia', 'Piotr']
```

Zagnieżdżone list comprehensions (często używane do macierzy):

Macierz 3x4 wypełniona zerami

```
macierz = [[0 for _ in range(4)] for _ in range(3)]
print(macierz)
# [[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
```

Tabliczka mnożenia

```
tabliczka = [[i * j for j in range(1, 11)] for i in range(1, 11)]
```

Zalety list comprehension:

- Bardzo czytelne
- Szybsze niż pętla for + append()
- Zajmuje mniej miejsca

Wady:

- Tworzy całą listę w pamięci od razu → problem przy bardzo dużych danych.

2. Generatory (funkcje z yield)

Generator to funkcja, która **zamiast *return* używa *yield*** Zwraca wartości „po kawałku” (po jednej na raz)

Różnica: return vs yield

zwykła funkcja:

```
def numbers():
    return [1, 2, 3]
```

zwraca wszystko naraz

generator:

```
def numbers():  
    yield 1  
    yield 2  
    yield 3
```

Gdy funkcja napotka **yield**, zatrzymuje swoje wykonanie i zwraca wartość. Przy następnym wywołaniu **next()** kontynuuje dokładnie od tego miejsca

```
gen = numbers()
```

```
print(next(gen))  
print(next(gen))  
print(next(gen))
```

```
for liczba in numbers():  
    print(liczba)
```

3. Generator expressions (wyrażenia generatorowe)

To „leniwa” wersja list comprehension. Zamiast `[]` używamy `()`.

```
# List comprehension (tworzy całą listę od razu)  
kwadraty_list = [x**2 for x in range(1000000)]
```

```
# Generator expression (tworzy generator – prawie zero pamięci)  
kwadraty_gen = (x**2 for x in range(1000000))
```

List comprehension (`[]`) – zachłanny (eager)

☞ tworzy całą listę od razu w pamięci

```
numbers = [x * 2 for x in range(5)]  
print(numbers)
```

☞ wynik:

```
[0, 2, 4, 6, 8]
```

☞ wszystko policzone natychmiast

Generator expression (()) – leniwy (lazy)

☞ nie liczy wszystkiego od razu

☞ generuje wartości dopiero kiedy są potrzebne

```
numbers = (x * 2 for x in range(5))  
print(numbers)
```

➡ wynik:

```
<generator object ...>
```

☞ jeszcze nic nie zostało policzone!

Generator działa dopiero przy użyciu:

```
for x in numbers:  
    print(x)
```

☞ albo:

```
next(numbers)
```

Zastosowanie

- Małe dane (do kilkudziesięciu tysięcy elementów) → możesz używać **list comprehension** (wygodniej).
- Duże dane lub kiedy przetwarzasz tylko raz → zdecydowanie **generator expression**.

match to nowoczesna **instrukcja porównywania wzorców** (pattern matching), dodana w Pythonie 3.10. Często nazywana jest „switch na sterydach”, bo jest o wiele potężniejsza niż klasyczny switch z innych języków.

Składnia:

```
match wartosc  
    case wzorzec1:  
        # kod gdy pasuje wzorzec1  
    case wzorzec2:  
        # kod gdy pasuje wzorzec2  
    case _:  
        # domyślny przypadek (jak else)
```

Przykład - ocena szkolna

```
ocena = "B"

match ocena:
    case "A":
        print("Celujący!")
    case "B":
        print("Bardzo dobry")
    case "C":
        print("Dobry")
    case "D":
        print("Dostateczny")
    case "E" | "F":          # | - or bitowy
        print("Niedostateczny")
    case _:
        print("Nieprawidłowa ocena")
```

KARTKÓWKA NA NASTEPNEJ LEKCJI (3 ostatnie lekcje)

Lekcja [Kartkówka]

Temat: Powtórzenie materiału do kartkówki

Zadanie 1

Napisz klasę Produkt, która będzie przechowywać informacje o produkcie w sklepie.

Wymagania:

1. Klasa ma posiadać:
 - a. Właściwość nazwę produktu (nazwa)
 - b. Właściwość cenę (cena)
2. Użyj @property, aby:
 - a. @property pobierać wartość ceny,
 - b. @cena.setter ustawiać nową cenę.
3. Setter ma sprawdzać:
 - a. cena nie może być mniejsza od 0.
4. Dodaj metodę info(), która wyświetli informacje o produkcie.

Rozwiązanie:

```

class Produkt():
    def __init__(self, nazwa, cena):
        self.nazwa_prodktu = nazwa
        self._cena = cena
    # get
    @property
    def cena(self):
        return self._cena

    # set
    @cena.setter
    def cena(self, value):
        if value < 0:
            raise ValueError("Cena nie może być ujemna")
        self._cena = value

    def info(self):
        print(f"Produkt: {self.nazwa_prodktu}, Cena: {self._cena} zł")

produkt1 = Produkt("Laptop", 3500)

produkt1.info()

print(produkt1.cena)

produkt1.cena = 4200
produkt1.info()

produkt1.cena = -100

print(produkt1.cena)

```

Zadanie 2

Napisz program wykorzystujący **dziedziczenie** oraz **polimorfizm**.

Utwórz klasę bazową **Pojazd**, która będzie zawierała metody:

- *poruszaj_sie()*

Metody w klasie bazowej mogą zawierać instrukcję **pass**.

Następnie utwórz klasy dziedziczące po klasie **Pojazd**:

- Samochod
- Rower

Każda klasa powinna:

- posiadać własny konstruktor `__init__`
- nadpisywać metody `poruszaj_sie()`

Na końcu programu:

1. Utwórz listę obiektów różnych klas.
2. Wykorzystaj pętlę `for`, aby wywołać metody `poruszaj_sie()` dla każdego obiektu.
3. Zaprezentuj działanie polimorfizmu.

Rozwiązanie

```
class Pojazd:
    def poruszaj_sie(self) -> str: pass

class Samochod(Pojazd):

    def __init__(self, marka, predkosc):
        self.marka = marka
        self.predkosc = predkosc

    def poruszaj_sie(self):
        return f"Samochód {self.marka} jedzie z prędkością {self.predkosc} km/h"

class Rower(Pojazd):

    def __init__(self, typ):
        self.typ = typ

    def poruszaj_sie(self):
        return f"Rower typu {self.typ} jest napędzany pedałami"

# Polimorfizm
pojazdy = [
    Samochod("BMW", 180),
    Rower("górski")
]

for pojazd in pojazdy:
    print(pojazd.poruszaj_sie())
    print("---")
```

Zadanie 3

Napisz program wykorzystujący **lis t comprehension**.

Utwórz listę liczb od 0 do 9, a następnie dodaj 1 do każdego elementu listy przy użyciu składni list comprehension.

```
plusJeden = [a+1 for a in range(10)]
```

```
print(plusJeden)
```

Lekcja

Temat: Powtórzenie z list, krotek, słowników, zbiorów oraz ich użycia wewnątrz klas

1. Mutowalność (mutable) vs niemutowalność (immutable)

Typ	Mutowalność	Czy można zmienić po utworzeniu?
list	mutowalna	✅ można dodawać, usuwać, zmieniać elementy
dict	mutowalny	✅ można dodawać, usuwać, zmieniać pary klucz-wartość
set	mutowalny	✅ można dodawać, usuwać elementy
tuple	niemutowalna	❌ NIE można zmienić! Jest jak "zamrożona lista"

1. List (lista) – MUTOWALNA

```
my_list = [1, 2, 3]
```

```
my_list[0] = 100      # zmiana elementu
my_list.append(4)      # dodanie elementu
my_list.pop(0)         # usuwa 1 element
my_list.remove(3)      # Usunięcie elementu po wartości
del my_list[0]         # Usunięcie przez del
my_list.clear()        # Wyczyszczenie całej listy
```

```
print(my_list)        # [100, 2, 3, 4]
```

2. Dict (słownik) – MUTOWALNY

```
my_dict = {"a": 1, "b": 2}
```

```

my_dict["a"] = 100          # zmiana wartości
my_dict["c"] = 3            # dodanie nowego klucza
my_dict.pop("a", None)      # Usunięcie elementu (bezpieczne – jak discard)
my_dict.pop("b")            # usunięcie po kluczu
del my_dict["c"]            # Usunięcie przez del
my_dict.clear()             # Wyczyszczenie słownika

print(my_dict)              # {'a': 100, 'b': 2, 'c': 3}

```

3. Set (zbiór) – MUTOWALNY

```

my_set = {1, 2, 3}

my_set.add(4)
my_set.remove(2)            # usunięcie z błędem
my_set.discard(3)          # usunięcie bez błędu
my_set.pop()               # usuwa losowy element
my_set.clear()              # wyczyszczenie

print(my_set)              # {1, 3, 4}

```

4. Tuple (krotka) – NIEMUTOWALNA

```

my_tuple = (1, 2, 3)

# my_tuple[0] = 10 # BŁĄD TypeError: 'tuple' object does not support item
# assignment

del my_tuple                # Możesz usunąć tylko całą krotkę:
my_tuple.append(4)          # brak takiej metody
my_tuple.clear()            # Nie istnieje clear() dla tuple.

```

Ważny przypadek (tuple z listą w środku)

```

my_tuple = (1, [2, 3])

my_tuple[1].append(4)

print(my_tuple)            # (1, [2, 3, 4])

```

Krotka jest niemutowalna, ale **obiekt w środku (lista)** już nie.

2. Podstawowe metody dla każdego typu

LISTA (mutowalna)

```
lista = [1, 2, 3]

# Dodawanie
lista.append(4)      # [1, 2, 3, 4] - dodaje na koniec
lista.insert(1, 99)  # [1, 99, 2, 3, 4] - wstawia na pozycję
lista.extend([5, 6]) # [1, 99, 2, 3, 4, 5, 6] - łączy listy

# Usuwanie
lista.remove(99)     # [1, 2, 3, 4, 5, 6] - usuwa pierwsze wystąpienie
element = lista.pop() # usuwa i zwraca ostatni (6)
lista.pop(0)         # usuwa i zwraca element z indeksu 0 (1)
lista.clear()        # [] - usuwa wszystko

# Inne przydatne
lista = [3, 1, 4, 1, 5]
lista.sort()         # [1, 1, 3, 4, 5] - sortuje
lista.reverse()      # [5, 4, 3, 1, 1] - odwraca
len(lista)           # 5 - długość
3 in lista            # True - sprawdza czy istnieje
```

KROTKA (niemutowalna) - mało metod

```
krotka = (1, 2, 3, 2, 4)

# Nie można dodawać ani usuwać!
# Ale można odczytywać:
print(krotka[0])      # 1
print(krotka.count(2)) # 2 - ile razy występuje
print(krotka.index(3)) # 2 - na którym indeksie jest 3
print(len(krotka))    # 5

# Jedyny sposób "dodania" - stworzenie nowej
nowa = krotka + (5, 6) # (1, 2, 3, 2, 4, 5, 6)
```

SŁOWNIK (mutowalny)

```
sloownik = {"a": 1, "b": 2}

# Dodawanie / zmiana
sloownik["c"] = 3          # {"a":1, "b":2, "c":3}
sloownik.update({"d": 4}) # dodaje wiele

# Usuwanie
del sloownik["b"]          # {"a":1, "c":3, "d":4}
sloownik.pop("c")          # usuwa i zwraca wartosc 3 dla klucza c
sloownik.popitem()         # usuwa i zwraca ostatnią parę ('d', 4)
sloownik.clear()           # {}

# Dostęp
print(sloownik.get("a", 0)) # 1 (bez błędu jeśli nie ma)
print(sloownik.keys())      # widok kluczy
print(sloownik.values())    # widok wartości
print(sloownik.items())     # widok par (klucz, wartość)
```

ZBIÓR (mutowalny) - elementy unikalne, nieuporządkowane

```
zbior = {1, 2, 3}

# Dodawanie
zbior.add(4)          # {1, 2, 3, 4}
zbior.add(2)          # {1, 2, 3, 4} - nic się nie dzieje (już jest)

# Usuwanie
zbior.remove(3)        # {1, 2, 4} - błąd jeśli nie ma
zbior.discard(10)      # {1, 2, 4} - brak błędu jeśli nie ma
zbior.pop()            # usuwa i zwraca jakiś element (nie wiadomo który)
zbior.clear()          # {}

# Operacje na zbiorach
A = {1, 2, 3}
B = {3, 4, 5}
print(A | B)           # {1,2,3,4,5} - suma
print(A & B)           # {3} - przecięcie
print(A - B)           # {1,2} - różnica
```


3. Proste klasy z zastosowaniem każdego typu

Klasa 1: ListaZakupow - używa LISTY

```
class ListaZakupow:
    def __init__(self):
        self.produkty = [] # Lista przechowuje produkty

    def dodaj(self, produkt):
        self.produkty.append(produkt)
        print(f"Dodano: {produkt}")

    def usun(self, produkt):
        if produkt in self.produkty:
            self.produkty.remove(produkt)
            print(f"Usunięto: {produkt}")
        else:
            print(f"Brak {produkt} na liście")

    def wyswietl(self):
        if not self.produkty:
            print("Lista zakupów jest pusta")
        else:
            print("LISTA ZAKUPÓW:")
            for i, produkt in enumerate(self.produkty, 1):
                print(f" {i}. {produkt}")

    def ile_produktow(self):
        return len(self.produkty)

# Użycie:
lista = ListaZakupow()
lista.dodaj("chleb")
lista.dodaj("mleko")
lista.dodaj("jajka")
lista.wyswietl()
lista.usun("mleko")
lista.wyswietl()
print(f"Liczba produktów: {lista.ile_produktow()}")
```

Klasa 2: Punkt2D - używa KROTKI (niemutowalna)

```
class Punkt2D:
    def __init__(self, x, y):
        # krotka przechowuje współrzędne (nie mogą się zmienić)
        self.wspolrzedne = (x, y)

    def pobierz_x(self):
        return self.wspolrzedne[0]

    def pobierz_y(self):
        return self.wspolrzedne[1]

    def odleglosc_od_poczatku(self):
        x, y = self.wspolrzedne
        return (x**2 + y**2) ** 0.5

    def __str__(self):
        return f"Punkt{self.wspolrzedne}"

# Nie ma metody do zmiany współrzędnych! (bo krotka jest niemutowalna)
# Aby zmienić, trzeba stworzyć nowy obiekt

# Użycie:
p1 = Punkt2D(3, 4)
print(p1)                # Punkt(3, 4)
print(f"x = {p1.pobierz_x()}") # x = 3
print(f"Odległość: {p1.odleglosc_od_poczatku():.2f}") # 5.00

# Nie można zrobić: p1.wspolrzedne[0] = 5 (błąd!)
# Aby zmienić, tworzymy nowy punkt:
p2 = Punkt2D(5, 6)
```

Klasa 3: Telefon - używa SŁOWNIKA

```
class Telefon:
    def __init__(self):
        # słownik: nazwa kontaktu -> numer telefonu
        self.kontakty = {}

    def dodaj_kontakt(self, nazwa, numer):
        self.kontakty[nazwa] = numer
        print(f"Dodano: {nazwa} -> {numer}")
```

```

def usun_kontakt(self, nazwa):
    if nazwa in self.kontakty:
        usuniety = self.kontakty.pop(nazwa)
        print(f"Usunięto: {nazwa} ({usuniety})")
    else:
        print(f"Brak kontaktu: {nazwa}")

def zadzwon(self, nazwa):
    if nazwa in self.kontakty:
        print(f"Dzwonię do {nazwa} pod numer {self.kontakty[nazwa]}")
    else:
        print(f"Nie mam numeru do {nazwa}")

def wyswietl_kontakty(self):
    if not self.kontakty:
        print("Brak kontaktów")
    else:
        print("KONTAKTY:")
        for nazwa, numer in self.kontakty.items():
            print(f" {nazwa}: {numer}")

def znajdz_po_numerze(self, numer):
    # szukamy kto ma dany numer
    for nazwa, nr in self.kontakty.items():
        if nr == numer:
            return nazwa
    return None

# Użycie:
telefon = Telefon()
telefon.dodaj_kontakt("Ania", "123-456-789")
telefon.dodaj_kontakt("Kamil", "987-654-321")
telefon.wyswietl_kontakty()
telefon.zadzwon("Ania")
kto = telefon.znajdz_po_numerze("987-654-321")
print(f"Numer 987-654-321 należy do: {kto}")

```

Klasa 4: LiczbyUnikalne - używa ZBIORU

```

class LiczbyUnikalne:
    def __init__(self):
        self.liczby = set() # zbiór przechowuje unikalne liczby

```

```

def dodaj(self, liczba):
    if liczba in self.liczby:
        print(f"{liczba} już jest w zbiorze!")
    else:
        self.liczby.add(liczba)
        print(f"Dodano: {liczba}")

def usun(self, liczba):
    if liczba in self.liczby:
        self.liczby.remove(liczba)
        print(f"Usunięto: {liczba}")
    else:
        print(f"{liczba} nie ma w zbiorze")

def wyswietl(self):
    if not self.liczby:
        print("Zbiór jest pusty")
    else:
        print(f"Zbiór liczb: {sorted(self.liczby)}") # sortujemy dla czytelności

def ile_liczb(self):
    return len(self.liczby)

def czy_zawiera(self, liczba):
    return liczba in self.liczby

def polacz(self, inny_zbior):
    """Łączy z innym zbiorem (suma)"""
    self.liczby.update(inny_zbior)
    print("Połączono zbiory")

# Użycie:
zb = LiczbyUnikalne()
zb.dodaj(5)
zb.dodaj(10)
zb.dodaj(5)      # komunikat: już jest!
zb.dodaj(7)
zb.wyswietl()    # {5, 7, 10}
zb.usun(10)
zb.wyswietl()    # {5, 7}
print(f"Czy jest 5? {zb.czy_zawiera(5)}")

```

4. Ćwiczenia dla uczniów

Ćwiczenie 1 (Lista)

Dodaj do ListaZakupow metodę `usun_ostatni()` która usuwa ostatni produkt z listy.

Ćwiczenie 2 (Krotka)

Stwórz klasę `KartaPlatnicza` która przechowuje w krotce: numer karty, datę ważności, kod CVV. Nie dodawaj metody do zmiany tych danych.

Ćwiczenie 3 (Słownik)

Dodaj do `Telefon` metodę `edytuj_numer(nazwa, nowy_numer)` która zmienia numer istniejącego kontaktu.

Ćwiczenie 4 (Zbiór)

Dodaj do `LiczbyUnikalne` metodę `czesc_wspolna(inny_zbior)` która zwraca nowy obiekt `LiczbyUnikalne` zawierający tylko liczby występujące w obu zbiorach.